

5교시 · MCP — AI에게 도구를 할당하는 표준

4교시의 `agent-lab` 앱을 그대로 사용.

문제 제기

실무에 필요한 도구는 3~4개로 안 끝난다 (문서 검색, 일정, 장비 이력, 티켓 등록...). 일이 두 갈래로 갈린다:

- 도구를 만드는 일 — 무엇을 할지 정하고 코드 작성 (누군가 어차피 해야 함)
- 도구를 연결하는 일 — 만들어진 걸 내 AI가 쓸 수 있게 연결

연결 방법을 매번 새로 짜면 도구가 늘수록 연결에만 시간이 든다 → 연결 규격을 하나로 통일하면?

MCP (Model Context Protocol)

- **Host** — 도구를 쓰는 쪽 (오늘은 `agent-lab`)
- **Server** — 도구를 내주는 쪽 (누가 만들었든 상관없음)

핵심: 서버가 먼저 "나는 이런 걸 할 수 있다"고 광고한다 (이름·설명문·인자 목록). Host는 그 목록을 받아 자기 도구 목록에 그대로 합친다. 그다음은 4교시와 동일 — AI는 합쳐진 목록의 설명문을 읽고 무엇을 부를지 정한다. 도구가 내 앱 안에 있든 남의 서버에 있든 판단 방식은 같다.

(참고: 서버는 도구뿐 아니라 참고자료·지시문 템플릿도 같은 방식으로 내줄 수 있음)

실습 — 서버 연결

1. **연결 전 확인:** `[도구]` 탭에서 사내 자료 질문 3개를 던져 답변/부른 도구를 기록 (아직 관련 도구 없음)
2. **연결:** `[MCP]` 탭 → 자료 조사 서버 `[연결]` 클릭 → `[지금 쓸 수 있는 도구]` 목록에 새 도구 추가됨(어느 서버 출신인지 표시)
3. **재확인:** 같은 질문 3개 재실행 → 호출 로그에서 실제 반환값과 답변 대조

이 서버는 어디서 도는가

자료 조사 서버는 내 PC에서 별도 프로세스로 돈다.

```
Get-Process python # 연결 상태 vs 연결 끊은 상태에서 프로세스 수 비교
```

`[MCP]` 탭에서 끊기/연결을 반복하며 숫자 변화 관찰 → 연결 = 따로 뜯 프로그램과 앱 사이를 잇는 것.

성격이 다른 서버 — Microsoft Learn 문서 서버

- **자료 조사 서버:** 내 PC에서 파이썬으로 돔 (인터넷 불필요)
- **Microsoft Learn 서버:** 인터넷 저편, MS가 운영, 우리가 만들지 않았고 안을 볼 수도 없음

- 하지만 붙이는 절차는 동일 — 버튼 하나.

`Get-Process python` 으로 확인하면 이번엔 프로세스 수가 늘지 않음(원격이라 로컬 프로세스가 안 뜸).

갈림길 테스트: 두 서버 모두에 걸릴 법한 질문(`문서에서 교정 주기를 찾아줘`)을 던져 어느 쪽이 선택되는지, 왜인지 두 도구의 설명문에서 근거를 찾아본다. ("내 PC 자료에서" 같은 조건을 넣으면 결과가 어떻게 바뀌는지도 확인)

코드로 보는 연결 — 등록과 통신

`mcp_catalog.json` :

```
{
  "doc-search": {
    "label": "자료 조사 서버",
    "spec": { "command": "python", "args": ["mcp-servers/doc-search/server.py"] }
  },
  "mslearn": {
    "label": "Microsoft Learn 문서 서버",
    "spec": { "url": "https://learn.microsoft.com/api/mcp" }
  }
}
```

로컬 서버는 실행 명령(`command`), 원격 서버는 주소(`url`)를 적는다.

[연결] 시 오가는 JSON-RPC 메시지 (요약):

앱 → {"method": "initialize", ...}	인사
서버 ← {"result": {"serverInfo": {"name": "doc-search"}}	자기소개
앱 → {"method": "tools/list"}	무엇을 할 수 있나
서버 ← {"result": {"tools": [{"name": "doc_search", "description": "...", "inputSchema": {...}]}}	도구 광고
앱 → {"method": "tools/call", "params": {"name": "doc_search", "arguments": {"query": "교정"}}}	실행 요청
서버 ← {"result": {"content": [{"type": "text", "text": "...}]}}	결과

"도구 광고"의 모양(이름·설명문·인자 스키마)이 4교시의 도구 선언과 동일한 꼴 — Host는 이것 받아 내장 도구 목록에 그대로 합칠 뿐이고, 그다음 AI 판단 방식은 똑같다. 전송 방식(로컬 프로세스 vs HTTP)만 다를 뿐 세 마디(인사→광고→호출)는 같다.

- [연결] = `mcp_config.json` 에 등록 → (로컬이면 프로세스 기동 후) 인사→광고 수신→도구목록 합침
- [끊기] = 설정에서 제거 → 도구가 목록에서 사라짐, 로컬이면 프로세스도 종료. 목록에 없는 도구는 AI가 부를 수 없음.

개발 — 원격 서버 직접 등록하기 (Tavily 웹 검색)

API Key가 필요한 첫 사례.

1. `app.tavily.com` 가입(구글 계정 가능) → `tvly-` 로 시작하는 Key 발급 → `.env` 에 추가:

`TAVILY_API_KEY=발급받은키`

2. 카탈로그 등록 요청:

이 앱의 MCP 카탈로그(`mcp_catalog.json`)에 Tavily 웹 검색 서버를 추가해 줘. - 원격 HTTP 서버이고, 주소 형식은 `https://mcp.tavily.com/mcp/?tavilyApiKey=<API키>` 아. - API 키는 `.env` 의 `TAVILY_API_KEY` 에서 읽어서 채워줘. 카탈로그 파일에 키가 그대로 적히면 안 돼. - 화면의 서버 목록에 "Tavily 웹 검색" 으로 보이게 해 줘.

(앱 재시작 필요)

3. [MCP] 에서 연결 → `오늘 한국의 주요 뉴스 알려줘` 로 테스트. 설명문이 영어로 온 것도 확인 — 남이 쓴 광고를 그대로 받은 것.

4. Tavily는 연결이 아니라 호출 시점에 키를 검사(Unauthorized 뜨면 `.env` 확인). 확인 끝나면 연결 끊어두기 (카탈로그엔 남아있으므로 필요시 재연결만 하면 됨).

정리 — 붙이면 곧 능력이다

- 서버는 자기가 무엇을 할 수 있는지 먼저 알려준다. Host는 그 목록을 받아 합친다.
- 성격이 전혀 다른 두 서버가 같은 절차로 붙었다.
- 붙인 서버는 앱 안에 있지 않다 — 따로 떠 있고, 우리는 그 사이를 연결했을 뿐.
- 연결은 곧 능력이다 → 무엇을 붙이느냐가 권한 문제가 된다. 오늘 붙인 서버는 읽기 전용이지만, 삭제 도구를 내주는 서버도 똑같이 버튼 하나로 붙는다.

두 겹 방어 (4교시 복습 + 확장)

- 역할 문구 — 텍스트로 막음. 편하지만 표현이 바뀌면 새어나갈 수 있음
- 연결 자체를 끊기 — 아예 안 붙이면 목록에 없음. 없는 도구는 부를 수 없음 (더 확실)
→ 필요한 것만 붙이고, 붙인 것 안에서 다시 역할로 좁히는 게 순서.

지금 가진 것: `agent-lab` — 내장 도구 + 자료 조사 서버 + Microsoft Learn 서버 (+ 카탈로그에 등록해둔 Tavily, 지금은 끊어둠)

다음 질문: 도구도 붙었고 문서도 읽힌다. 그런데 일이 커지면 하나가 다 감당할 수 있을까? → 6교시 Multi-Agent